

Coding Standards – V.A.P.O.R.

Vehicle Analytics, Processing and Operations in Real-time

Team: Kilimanjaro StoneCap

Project: Vehicle Analytics Platform

1. Overview

This document outlines the coding standards and conventions for the V.A.P.O.R. system. These standards ensure uniformity, clarity, flexibility, reliability, and efficiency across our multi-component system.

2. Naming Standards

Variables & Constants

- camelCase for variables and non-React functions
- UPPER_SNAKE_CASE for constants and environment keys
- Variable names should be meaningful and intention-revealing
 - Avoid single letters except for conventional iterators
- Types, Interfaces & Enums (JavaScript/ JSDoc where used)
- PascalCase for type/shape definitions (e.g. VehicleRecord, TelemetryEvent)

React Components & Hooks

- PascalCase for component files and component names (e.g. FleetDashboard.jsx, LiveVehicleMap.jsx)
- camelCase prefixed with use for custom hooks (e.g. useVehicleStore.js, useMobile.js)

Files & Folders

- PascalCase for React component files (Dashboard.jsx)
- Lowercase/kebab-case for folders (components, services, map, dashboard)
- CSS/Tailwind config files mirror their component where co-located

Database

- Snake_case for table and column names (e.g. vehicle_id, clean_telemetry)
- Table names are plural (vehicles, trips)
- Migration files follow V{n}__description.sql

3. Code Layout & Structure

Formatting & Indentation

- 2 spaces for indentation
- Use Prettier for formatting and EditorConfig to harmonise indentation/line endings
- Line length target 100-120 characters; wrap long JSX props and object literals vertically.

Imports

- Order: Node/polyfills -> external packages -> absolute aliases -> relative imports, grouped with a blank line between groups
- No circular imports; prefer passing dependencies as parameters over cross-module singletons

Comments

- Explanations should be kept brief
- Use for document side effects and non-obvious logic (e.g. why a threshold or window size was chosen); avoid redundant comments

One Statement per Line

- Avoid chaining unrelated expressions on one line

React/TS Conventions

- Prefer function components with hooks
- Keep components under ~200 lines; extract custom hooks for reusable logic
- Side effects go in `useEffect` with complete dependency arrays; clean up sockets/intervals/timers in the return callback.
- Use `useCallback/useMemo` to stabilise handlers and computations passed to children
- Zustand stores are the single source of truth for cross-component state (fleet data, live positions, auth); avoid prop drilling

CSS & Theming

- TailwindCSS utility classes as default; avoid hand-rolled CSS unless Tailwind can't express it
- Design Tokens (colours, spacing) defined in `tailwind.config` rather than hard-coded hex values
- Desktop-first responsive design

Accessibility & Testability

- Use semantic elements (buttons, nav, section) and ARIA where needed
- All interactive elements must be keyboard accessible
- Stable test selectors: `data-cy="..."` is the only E2E selector mechanism, applied consistently across dashboard/map/auth components

4. Error Handling & Logging

Principles

- Fail fast near the source; recover gracefully where user experience matters.
- Never leak secret logs (tokens, DB credentials, Cognito secrets, PII)
- Provide user-friendly messages in the UI; keep technical detail in logs/CloudWatch

Frontend

- Wrap API calls in `try/catch`; surface toasts/ notifications for transient issues (failed fetch, socket disconnect from live map)

- Centralise error handling in frontend/src/services
- Use a React Error Boundary for rendering-level crashes

Backend (Express)

- Error responses go through the shared error (response, message, statusCode) helper in utils/response.js. Handlers and middleware call this directly rather than throwing to a global error handler
- Keep messages user-safe (no stack traces or internal detail); log the real error server-side via console.error with context (e.g. which module/action failed)
- Choose the correct HTTP status per failure type (401 for auth failures, 403 for permission/role failures, 404 for missing resources, etc.)

Lambdas

- Wrap Kinesis record processing in try/catch per-record so a single malformed record doesn't fail the whole batch
- Log processing failures to CloudWatch with the offending factors for traceability
- Never let a Lambda fail silently. Failed records should be identifiable after the fact

Validation

- Server: validate all request bodies/params before business logic
- Client: validate forms; sanitise any interpolated values used in the DOM

5. Configuration & Secrets

These include Naming and local conventions only. Full secrets management strategy is documented in the SAS 3.2.3 Deployment Requirements

- Environment files: use checked-in templates(.env.example) for both frontend/.env and backend.env; never commit real secrets
- Frontend env keys prefixed VITE_... (e.g. VITE_API_URL, VITE_MAPBOX_TOKEN)
- Backend secrets (DB credentials, Cognito pool/client IDs, AWS region) loaded from environment variables, never hard coded
- No secrets committed to version control at any point. This is enforced via .gitignore and code review

6. General Principles

Core Values

1. Clarity over Cleverness – Write code that tells a story
2. Consistency – Follow established patterns throughout the codebase
3. Modularity – Clear separation between UI, business logic and data layers
4. Security First – Always consider security implications (especially around Cognito/JWT and telemetry ingestion)
5. Performance awareness – Write efficient code that scales to 50+ concurrently reporting vehicles

Code Style Philosophy

- Favour explicit over implicit
- Use meaningful names that describe intent
- Keep functions small and focused
- Minimise cognitive load for future development

7. File structure & Organisation

File Naming Conventions

- React Components: PascalCase (FleetDashboard.jsx)
- Pages: PascalCase (SettingsPage.jsx)
- Hooks: camelCase, use-prefixed (useMobile.js)
- Utilities: camelCase (formatTimestamp.js)
- CSS files (where not Tailwind-only): kebab-case
- Lambda handlers: snake_case folder per function (kinesis_telemetry_ingestion)
- SQL migrations: V{n}__description.sql
- Backend test suites: <module>.test.js, located in backend/__tests__/
- Backend test helper scripts (e.g. token generation): backend/tests/

8. Git repository and Strategy

Git Flow

Branch	Purpose
main	Production: stable, release ready code
develop	Integration: latest development changes
feature/*	Feature branches to implement specific features: branch from develop and merge back to develop
doc/*	Document branches to implement specific documentation: branch from develop and merge back to develop
fix/*	Fix branches to implement specific fixes on existing features: branch from develop and merge back to develop

All development work happens on feature/* or doc/* branches. No direct commits to main or develop

Feature Workflow

#1. Start from develop

```
git checkout develop
git pull origin develop
```

2. Create feature branch

```
git checkout -b feature/your-feature-name
```

3. Commit changes

```
git add .
git commit -m "feat: description of your feature"
```

4. Push to remote

git push origin feature/your-feature-name

5. Open a Pull Request from feature/* into develop

Branch Naming

- Descriptive, kebab-case: feature/live-vehicle-map, fix/integration-testing, doc/coding-standards
- Alphanumeric only; no consecutive hyphens

Commit Messages

- Conventional prefixes: feat:, fix:, docs:, test:, refactor:, chore:

Review Process

- CI runs linting first, then unit/integration tests, then SonarCloud analysis
- All 4 members approving review required before merge into develop
- main only receives merges from develop ahead of a demo release

Linting

- ESLint enforced across frontend, backend and lambdas
- SonarCloud quality gate must pass (no new blocker/ critical issues) before merge

9. Frontend Standards (React + Vite)

Component Organisation

- Functional components with hooks only
- Components grouped by domain under src/components (auth, dashboard, layout, map, shared, ui)
- Extract custom hooks for reusable logic (data fetching, socket subscriptions)
- Use React.memo() for performance-critical components (e.g. map markers re-rendering on every telemetry tick)

State Management

- Local component state: useState
- Cross cutting state (auth session, live fleet data, filters): Zustand stores under src/store/
- Avoid prop drilling; pull from relevant store instead

Data Visualisation

- Recharts for standard dashboard charts (KPIs, time-series)
- D3.js reserved for custom/non-standard visualisations Recharts can't express
- Mapbox GL JS for the live vehicle map; marker updates should be batched, not re-rendered per individual telemetry event

CSS Standards

- TailwindCSS utility-first; no inline styles except for values that must be computed at runtime (e.g. marker position)
- Consistent spacing via Tailwind's default scale
- Desktop-first responsive design

10. Backend Standards (Node.js + Express)

API Structure

- Organise code into controllers/, routes/, middleware/, db/, utils/ as per existing backend/src layout
- Shared error()/success() response helpers (utils/response.js) used consistently across all endpoints for a uniform response shape
- Implement proper input validation on every route
- Follow RESTful conventions; use meaningful HTTP status codes

Error Handling

- Custom error classes per error type (e.g. ValidationError, NotFoundError, AuthError)
- Global error-handler middleware as the single place responses is formatted
- Never expose internal error details (stack traces, DB errors) to clients

Security Standards

- Cognito-issued JWTs are validated on every protected route via middleware/auth.js (authenticate, requireRole)
- In production, the JWT signature/expiry is already validated by API Gateway; the backend decodes the payload to identify the user and checks is_active against the DB. In test environments, the middleware verifies signatures locally against JWT_SECRET
- Environment variables for all sensitive config (DB credentials, Cognito pool, client ID's)
- Parameterised queries only, no string concatenated SQL ever
- Input validation and sanitisation on all user-supplied data
- Rate limiting on public-facing endpoints

11. Data Pipeline Standards (AWS Lambda / Node.js)

Code Organisation

- One Lambda per responsibility (e.g. kinesis_telemetry_ingestion handles raw Kinesis records only)
- Use descriptive variable and function names; avoid single-letter variables outside of loop indices

Data Processing

- Validate incoming telemetry records before writing to the database; reject or quarantine malformed records rather than failing the whole batch.
- Handle missing or corrupt telemetry fields gracefully (e.g. missing GPS fix) without crashing the pipeline

- Use TimescaleDB-appropriate batch inserts for high throughput telemetry writes rather than row-by-row inserts
- Log processing steps and failure counts for debugging via CloudWatch

12. API Design Guidelines

RESTful Conventions

- Standard HTTP methods (GET, POST, PUT, DELETE)
- URLs structured hierarchically and consistently (/vehicles/:id/trips)
- Plural nouns for resource endpoints
- API Gateway path includes environment stage (/prod/...)
- Pagination on all list endpoints

Response Standards

- Consistent JSON response envelope (success, data/error, meta)
- Meaningful error messages, no leaked internals
- Appropriate HTTP status codes for every response path

13. Testing Conventions

These include file organisation and naming only. Coverage targets, test types and defect management processes are documented in the Testing policy document

- Backend test suites live in backend/ __tests__/, one file per module named <module>.test.js (e.g. auth_test.js, vehicles.test.js)
- Test helper/utility scripts (e.g. JWT generation for auth tests) live in backend/tests/, kept separate from the suits themselves
- Frontend: Jest for unit/integration (frontend/src/ __tests__), Cypress for E2E
- Stable selectors: data-testid="..." is the standard selector mechanism for both Jest and Cypress tests, applied consistently across components
- Mock external dependencies (Cognito, DB) in unit tests as in __mocks__/

14. Code Review Process

Review Requirements

- All pull requests require the approval of all 4 other members before merging into develop.
- Reviewers must check code quality, standards compliance and SonarCloud gate status
- Tests must be included and passing before review is requested

Review Guidelines

- Focus on clarity and maintainability, not personal style preference
- Suggest improvements constructively
- Verify naming, folder placement, and error-handling conventions are followed
- Check for potential security issues (unvalidated, exposed secrets, missing auth checks).

15. CI/CD Conventions

This is for pipeline stage ordering only. The full CI/CD pipeline diagram and deployment/rollback strategy are documented in the SAS, 3.2.3.

- Lint and format checks run on every commit
- Full Jest test suite executed before merge
- Coverage report generated and uploaded to Codecov
- SonarCloud static analysis run on every PR

16. Configuration Management

Environment Variables

- Never commit secrets to version control
- Consistent naming prefixes (VITE.. for frontend, plain UPPER_SNAKE_CASE for backend/Lambda)
- .env.example kept up to date as the source of truth for required variables

Best Practices

- Environment-specific configuration files/values per stage (dev/staging/prod)
- Graceful fallback only where safe (e.g. local dev defaults), never for production secrets
- Configuration kept separate from application code at all times

Conclusion

These coding standards ensure consistency, quality and maintainability across the V.A.P.O.R. platform. All team members are expected to follow these guidelines to maintain code quality and facilitate collaboration.

For questions about these standards, please contact the team lead at:

Kilimanjaro.capstone@gmail.com or raise an issue in the project repository